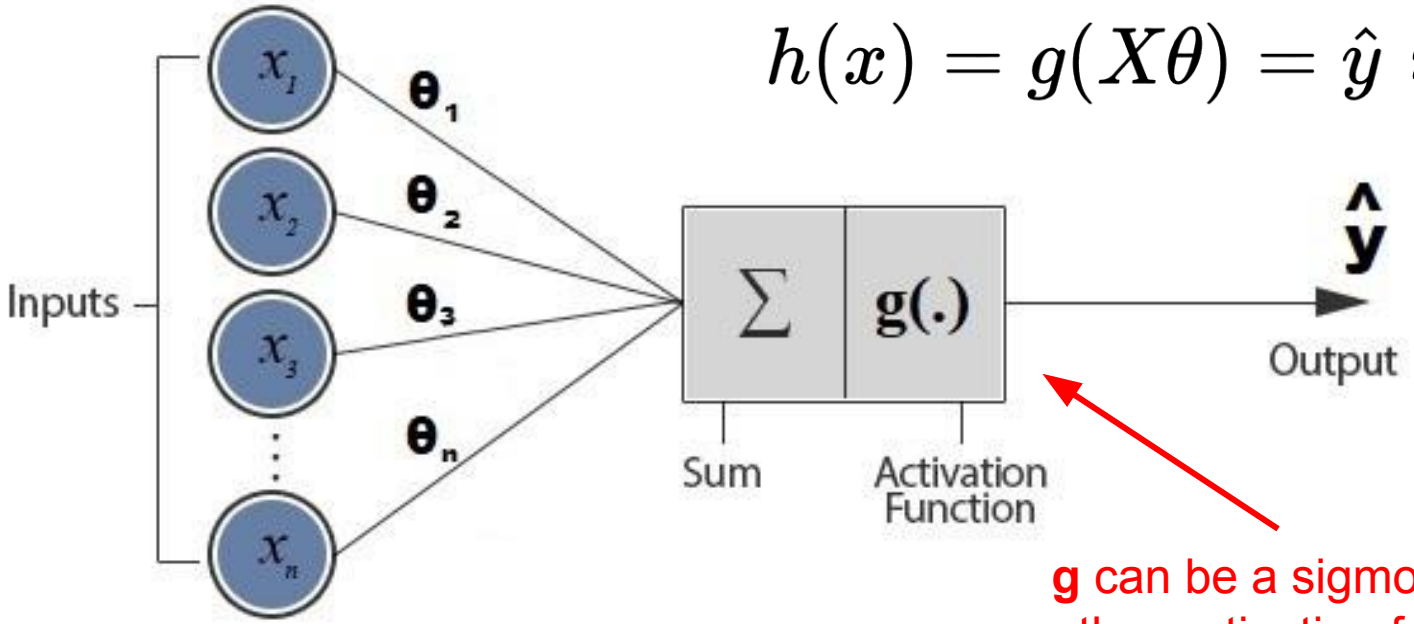
Deep Network Development

Lecture #6

Viktor Varga
Department of Artificial Intelligence, ELTE IK

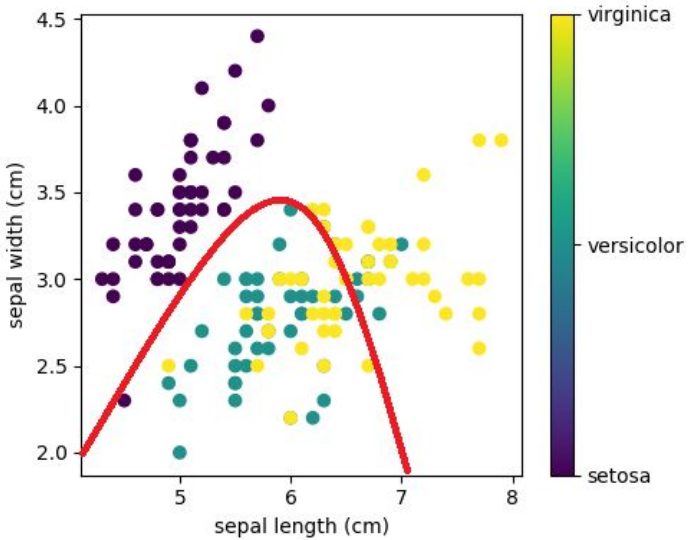
Last week - The artificial neuron model

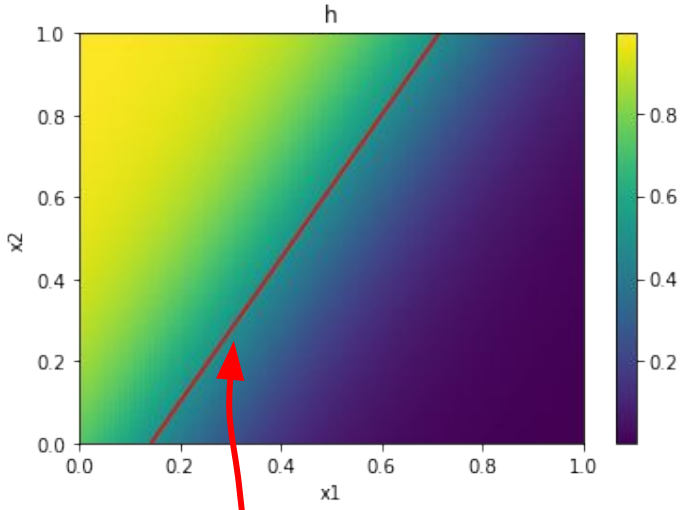


An artificial neuron is a (multivariate) **logistic regression if g is sigmoid!**

Last week - Multilayer Perceptron (MLP)

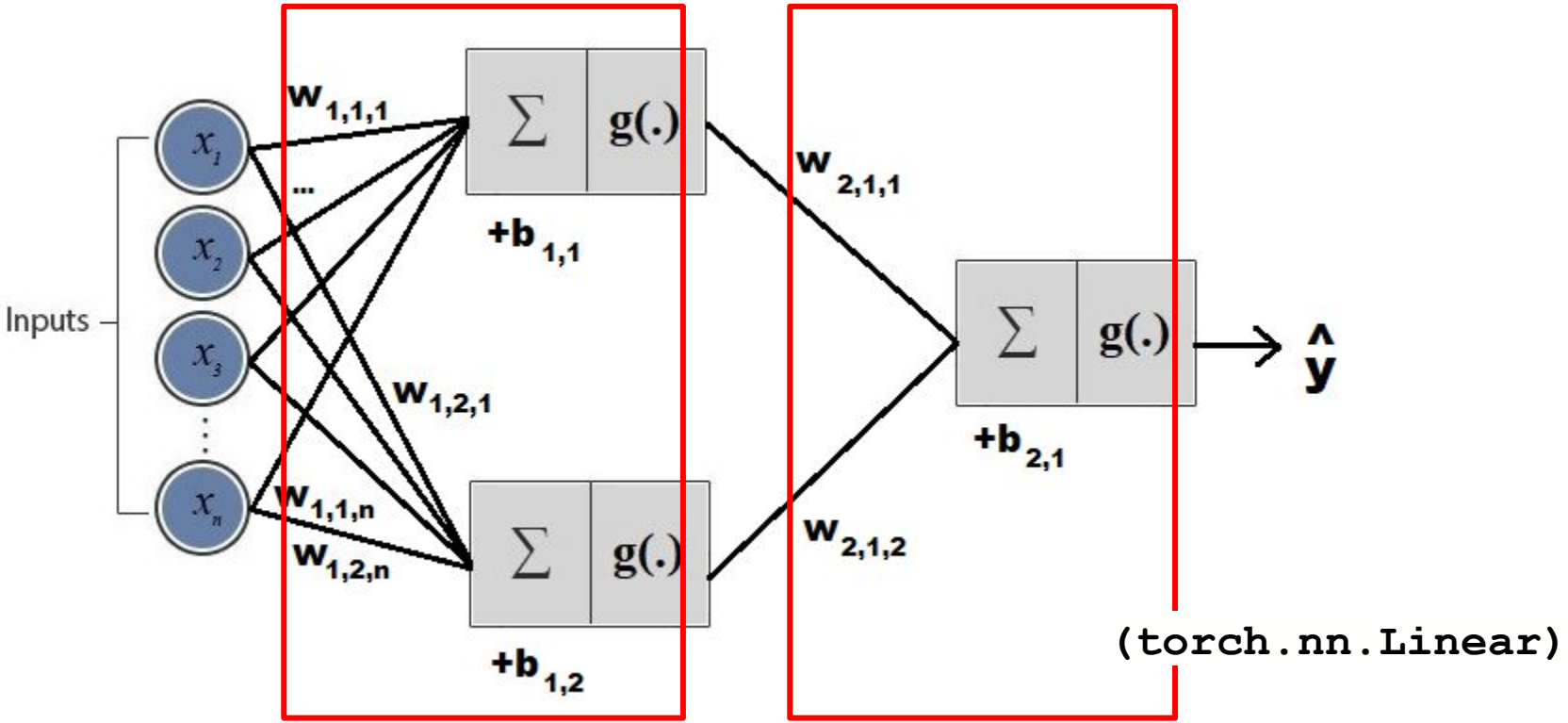
Most tasks cannot be solved by a single artificial neuron.





A linear decision boundary represented by **a single neuron is not capable of solving linearly inseparable problems**.

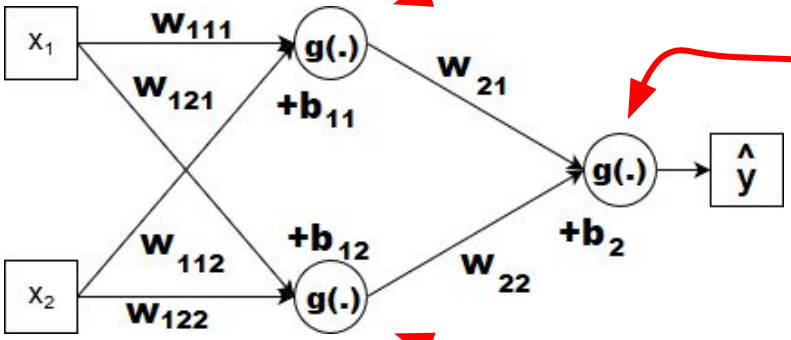
Last week - Multilayer Perceptron (MLP)



Fully connected / dense (neuron) **layers**

Last week - The expressive power of neural networks

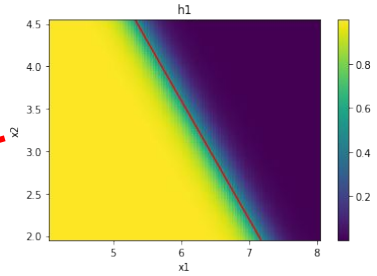
$$w_{111} = -7,\ w_{121} = -5,\ b_{11} = 60$$

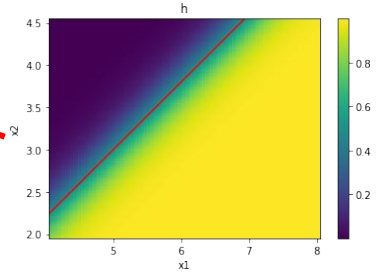


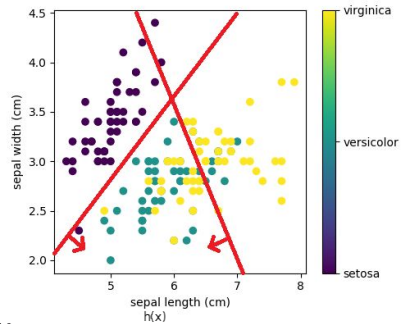
$$w_{21} = 7,\ w_{22} = 7,\ b_2 = -9$$

$$w_{112} = 4,\ w_{122} = -5,\ b_{12} = -5$$

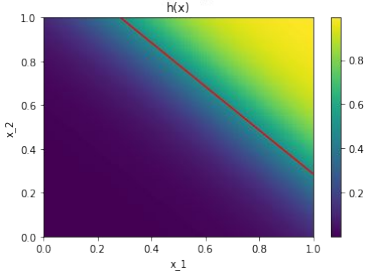
The weights were set manually here…



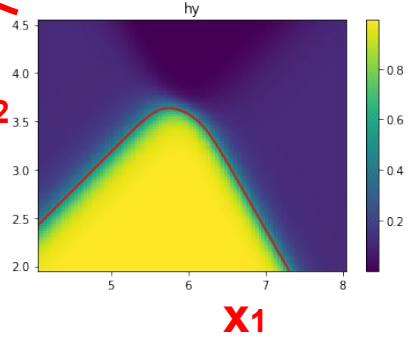




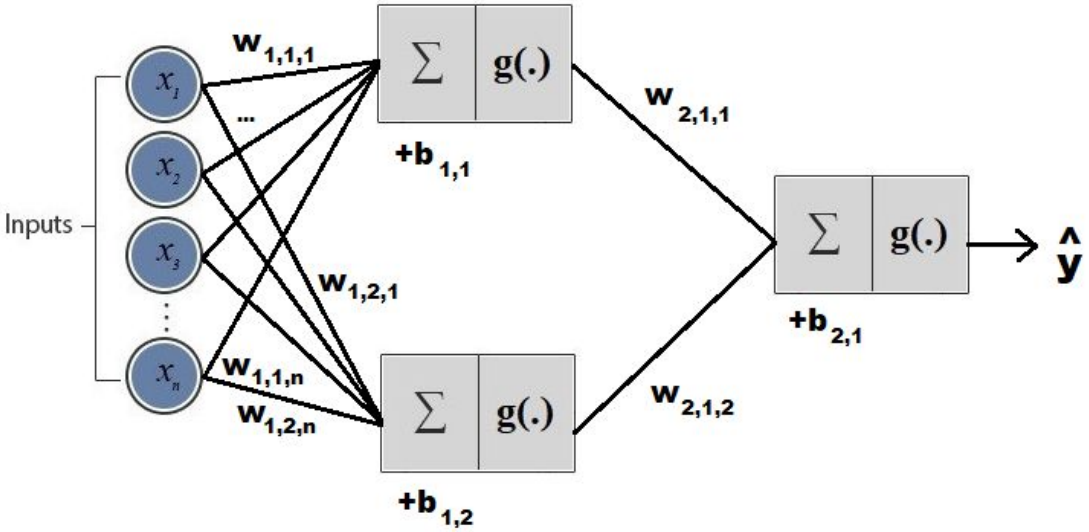
h2



h1



Last week - Multilayer Perceptron (MLP)



$$W_1 \in \mathbb{R}^{2\times n}$$
$$b_1 \in \mathbb{R}^2$$

$$W_2 \in \mathbb{R}^{1\times 2}$$
$$b_2 \in \mathbb{R}^1$$

New notation: Theta is the set of all weight matrices and bias vectors.
(i.e., the parameters)

$$\Theta = \{W_1, b_1, W_2, b_2\}$$

Last week - Multilayer Perceptron (MLP)

The hypothesis function of a two-layer MLP neural network:

$$h(x) = g_2(W_2\, g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

The output of the first layer

Loss functions:

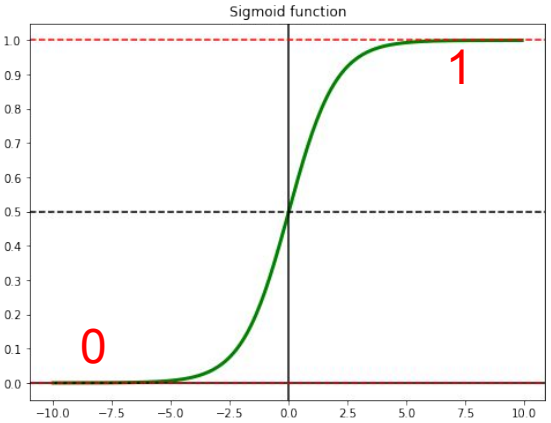
- **We always need intermediate activation functions** (nonlinearities) **in MLPs**
- **The last activation function depends on the task**

- **Regression:** MSE
- **Classification** (binary)**:** Logistic loss / Binary Cross-entropy (BCE)

Last week - Activation functions

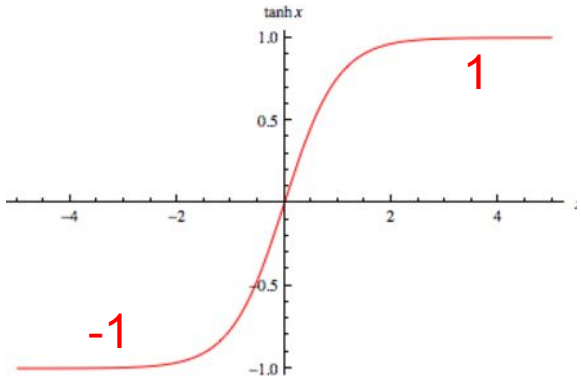
Popular activation functions

sigmoid



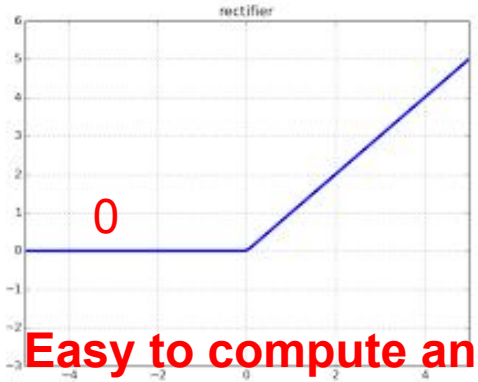
$$g(z) = \frac{1}{1+e^{-z}}$$

tanh



$$g(z) = \tanh(z) = \frac{e^{2x}-1}{e^{2x}+1}$$

ReLU
(Rectified Linear Unit)



Easy to compute and almost always works well

$$g(z) = \mathrm{ReLU}(z) = \max(0, z)$$

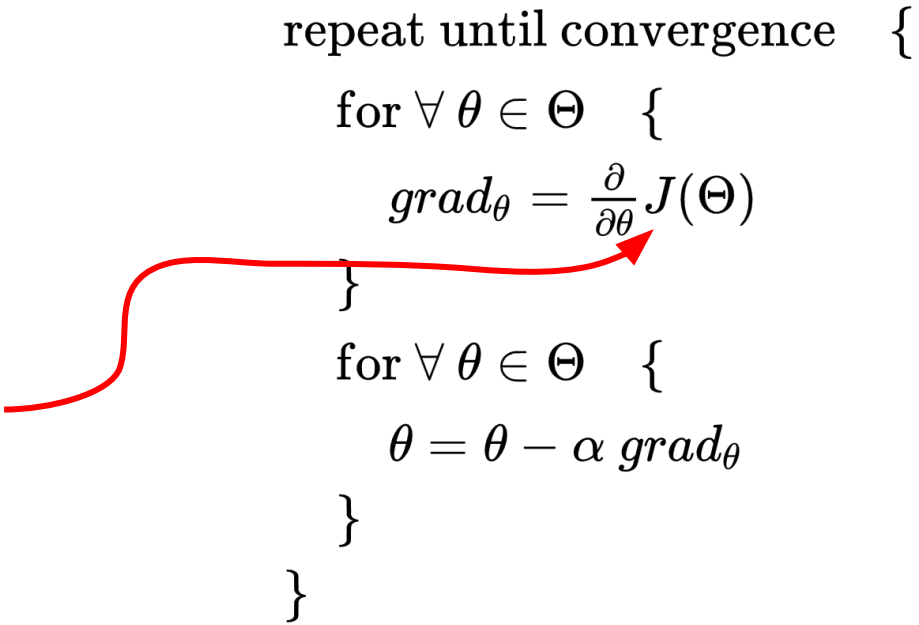
Last week - Training an MLP

“Batch” gradient descent

We average the loss over data points in the entire training set.

For example:

$$J(\theta) = \frac{1}{2m}\sum_{j=1}^{m}(h_\theta(x^{(j)}) - y^{(j)})^2$$



Last week - Training an MLP

“Batch” gradient descent

We average the loss over data points in the entire training set.

For example:

$$J(\theta) = \frac{1}{2m}\sum_{j=1}^{m}(h_\theta(x^{(j)}) - y^{(j)})^2$$

repeat until convergence {

  for $\forall\, \theta \in \Theta$ {

    $grad_\theta = \frac{\partial}{\partial\theta} J(\Theta)$

  }

  for $\forall\, \theta \in \Theta$ {

    $\theta = \theta - \alpha\, grad_\theta$

  }

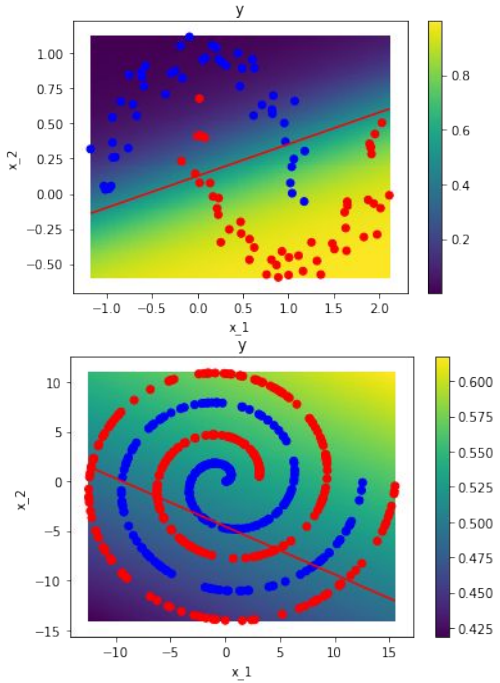
}

Usually more efficient: **Stochastic Gradient Descent** (SGD) **algorithm**
Compute gradients on a randomly selected sub-sample of the dataset (a mini-batch)
→ **Approximate, but much faster computation of the gradients.**

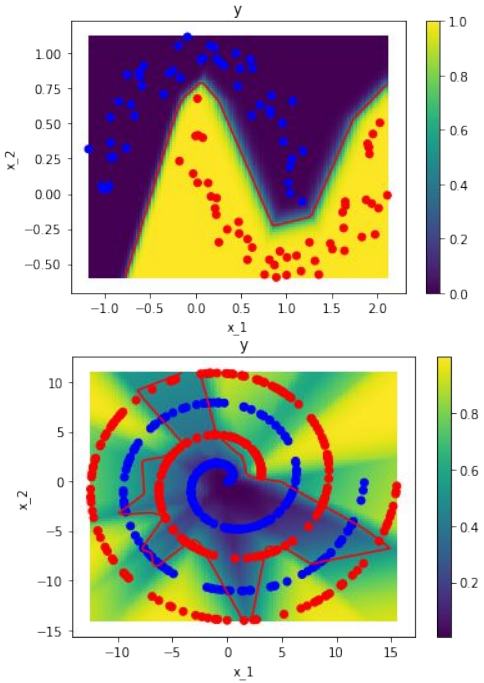
These weights were found using gradient descent.

Last week - The expressive power of neural networks

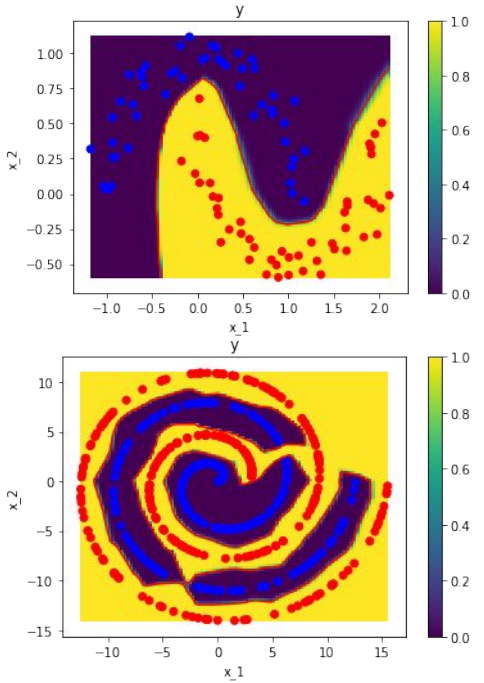
Logistic regression



MLP, 2 layers, 20+1 neurons



MLP, 4 layers, 20+20+20+1 neurons



Last week - Example tasks with multiple labels

A regression task with multiple label variables:

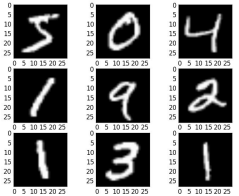
x_1: Weight of a patient
x_2: Age of a patient
x_3: Sex of a patient

→

y_1: Cholesterol levels of the patient
y_2: Blood sugar levels of the patient

A classification task with more than two categories (multi-class)**:**

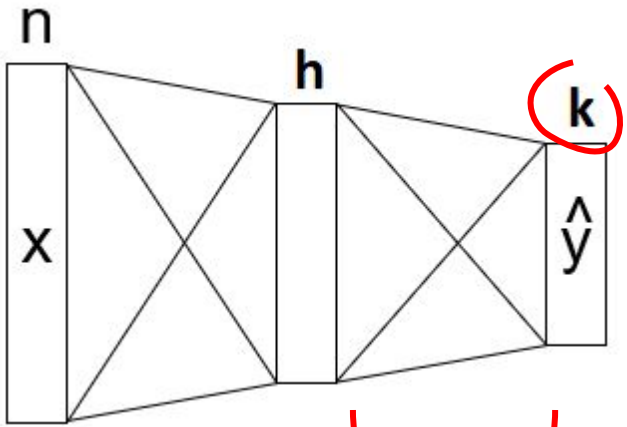
x_1, ..., x_{784} : Brightness of pixels →



y_1, ..., y_{10}: Probabilities of the digits from 0 to 9 being in the image

Last week - Multiple label variables

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$



$W_1 \in \mathbb{R}^{h \times n}$ $W_2 \in \mathbb{R}^{k \times h}$

$b_1 \in \mathbb{R}^h$ $b_2 \in \mathbb{R}^k$

Let y be a vector, similarly to x!

$$\Theta = \{W_1, b_1, W_2, b_2\}$$

Last week - Multiple label variables, **regression**

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

y hat and **y** are vectors

Last activation function: **None** (identity)

Loss: MSE, also averaged over the elements of the label vector

$$J(\Theta) = \frac{1}{2mk} \sum_{j=1}^{m} ||\hat{y}^{(j)} - y^{(j)}||_2^2 = \frac{1}{2mk} \sum_{j=1}^{m} \sum_{i=1}^{k} (\hat{y}_i^{(j)} - y_i^{(j)})^2$$

Last week - Multi-class **classification**

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

y hat and **y** are vectors

Last activation function: **Softmax**

Loss: Cross-entropy

$$J(\theta) = -\frac{1}{m} \sum_{j=1}^{m} \sum_{i=1}^{k} y_i log(\hat{y}_i)$$

one-hot

0
1
0
0

0.64
0.21
0.06
0.09

Training an MLP

We will use gradient descent…

$$
\begin{aligned}
&\text{repeat until convergence} \quad \{ \\
&\quad \text{for } \forall\, \theta \in \Theta \quad \{ \\
&\qquad grad_\theta = \frac{\partial}{\partial \theta} J(\Theta) \\
&\quad \} \\
&\quad \text{for } \forall\, \theta \in \Theta \quad \{ \\
&\qquad \theta = \theta - \alpha\, grad_\theta \\
&\quad \} \\
&\}
\end{aligned}
$$

Training an MLP

We will use gradient descent…

$$
\begin{aligned}
&\text{repeat until convergence} \quad \{ \\
&\quad \text{for } \forall\, \theta \in \Theta \quad \{ \\
&\quad\quad grad_\theta = \frac{\partial}{\partial \theta} J(\Theta) \\
&\quad \} \\
&\quad \text{for } \forall\, \theta \in \Theta \quad \{ \\
&\quad\quad \theta = \theta - \alpha\, grad_\theta \\
&\quad \} \\
&\}
\end{aligned}
$$

We need to calculate the derivative of the loss function w.r.t. each parameter. This gives us the gradient vector, which we use to update the parameters.

Let's expand the matrix form of the hypothesis function to make things easier!

Multilayer Perceptron (MLP) expanded

The hypothesis function of a two-layer MLP neural network:

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Expanded:

$$h(x) = w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) +$$
$$w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1} = \hat{y} \approx y$$

Multilayer Perceptron (MLP) expanded

The hypothesis function of a two-layer MLP neural network:

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Expanded:

$$h(x) = w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) +$$
$$w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1} = \hat{y} \approx y$$

$$J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (h(x^{(j)}) - y^{(j)})^2$$

$$\Theta = \{w_{1,1,1}, w_{1,1,2}, \ldots, b_{1,1}, \ldots\}$$

Multilayer Perceptron (MLP) expanded

We need to calculate the derivative of the loss function w.r.t. each parameter!

$$h(x) = w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) + w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1} = \hat{y} \approx y$$

$$J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (h(x^{(j)}) - y^{(j)})^2$$

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \; ?$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,2}} = \; ?$$

$$\cdots$$

Multilayer Perceptron (MLP) expanded

How do we calculate the derivative of composite functions?

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = ?$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,2}} = ?$$

$$\cdots$$

Chain rule

Reminder: Derivatives of composite functions

$$(f \circ g)' = (f' \circ g) \cdot g'$$

The same with **Leibniz-notation**:

$$\frac{\partial u}{\partial x} = \frac{\partial u}{\partial z} \cdot \frac{\partial z}{\partial x} \quad \text{where} \quad z := g(x),\ u := f(z)$$

Chain rule

Reminder: Derivatives of composite functions

$$(f \circ g)' = (f' \circ g) \cdot g'$$

The same with **Leibniz-notation**:

$$\frac{\partial u}{\partial x} = \frac{\partial u}{\partial z} \cdot \frac{\partial z}{\partial x} \quad \text{where} \quad z := g(x),\ u := f(z)$$

The derivative of the expression **u** with respect to **x**

The rule for differentiating composite functions is also known as the **chain rule**.

Chain rule - An example

Example: Calculate the derivative of the sigmoid function!

$$f(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{\partial f(x)}{\partial x} = \ ?$$

Chain rule - An example

Example: Calculate the derivative of the sigmoid function!

$$f(x) = \frac{1}{1 + e^{-x}} \qquad \frac{\partial f(x)}{\partial x} = \ ?$$

Let's define intermediate variables for easier derivation!

$$z := -x$$

$$q := 1 + e^{z}$$

$$f(x) := \frac{1}{q} = q^{-1}$$

Chain rule - An example

Chain rule: We calculate the derivatives of the intermediate variables with respect to the chained variable, then multiply the results together.

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x}$$

$$z := -x$$

$$q := 1 + e^z$$

$$f(x) := \frac{1}{q} = q^{-1}$$

Chain rule - An example

Chain rule: We calculate the derivatives of the intermediate variables with respect to the chained variable, then multiply the results together.

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x}$$

$$z := -x$$

$$q := 1 + e^z$$

$$f(x) := \frac{1}{q} = q^{-1}$$

Reminder: A list of rules to calculate symbolic derivatives
https://www.math.ucdavis.edu/~kouba/Math17BHWDIRECTORY/Derivatives.pdf

Chain rule - An example

Chain rule: We calculate the derivatives of the intermediate variables with respect to the chained variable, then multiply the results together.

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x}$$

$$z := -x$$

$$q := 1 + e^{z}$$

$$f(x) := \frac{1}{q} = q^{-1}$$

$$\frac{\partial z}{\partial x} = -1$$

$$\frac{\partial q}{\partial z} = e^{z} = e^{-x}$$

$$\frac{\partial f(x)}{\partial q} = -q^{-2} = -\frac{1}{(1+e^{-x})^2}$$

Chain rule - An example

Chain rule: We calculate the derivatives of the intermediate variables with respect to the chained variable, then multiply the results together.

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x} = (-1) \cdot (e^{-x}) \cdot -\frac{1}{(1+e^{-x})^2}$$

$$z := -x$$

$$q := 1 + e^z$$

$$f(x) := \frac{1}{q} = q^{-1}$$

$$\frac{\partial z}{\partial x} = -1$$

$$\frac{\partial q}{\partial z} = e^z = e^{-x}$$

$$\frac{\partial f(x)}{\partial q} = -q^{-2} = -\frac{1}{(1+e^{-x})^2}$$

Chain rule - An example

Example: Calculate the derivative of the sigmoid function!

$$f(x) = \frac{1}{1 + e^{-x}} \qquad \frac{\partial f(x)}{\partial x} = \; ?$$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x} = (-1) \cdot (e^{-x}) \cdot -\frac{1}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})^2}$$

Chain rule - An example

Example: Calculate the derivative of the sigmoid function!

$$f(x) = \frac{1}{1 + e^{-x}} \qquad \frac{\partial f(x)}{\partial x} = \; ?$$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial x} = (-1) \cdot (e^{-x}) \cdot -\frac{1}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})^2}$$

$$= \; \ldots \; = f(x)(1 - f(x))$$

Training a Multilayer Perceptron (MLP)

Back to MLP…

How do we calculate the derivative of composite functions?

$$J(\Theta) = \frac{1}{2m}\sum_{j=1}^{m}(h(x^{(j)}) - y^{(j)})^2$$

$$h(x) = w_{2,1,1}g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) + w_{2,1,2}g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1} = \hat{y} \approx y$$

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = ? \qquad \frac{\partial J(\Theta)}{\partial w_{2,1,1}} = ? \qquad \ldots$$

Training a Multilayer Perceptron (MLP)

Back to MLP…

How do we calculate the derivative of composite functions?

$$J(\Theta) = \frac{1}{2m}\sum_{j=1}^{m}(h(x^{(j)}) - y^{(j)})^2$$

$$h(x) = w_{2,1,1}g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) + w_{2,1,2}g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1} = \hat{y} \approx y$$

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = ? \qquad \frac{\partial J(\Theta)}{\partial w_{2,1,1}} = ? \qquad \ldots$$

Let’s define intermediate variables!

Training a Multilayer Perceptron (MLP)

Let's define intermediate variables!

$$J(\Theta) = \frac{1}{2m}\sum_{j=1}^{m}(h(x^{(j)}) - y^{(j)})^2$$

$$\hat{y} := h(x)$$

$$J(\Theta) = \frac{1}{2m}\sum_{j=1}^{m}(\hat{y} - y)^2$$

Training a Multilayer Perceptron (MLP)

Let's define intermediate variables!

$$\hat{y} = w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}) + w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}) + b_{2,1}$$

$$z_1 := w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1})$$

$$z_2 := w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2})$$

$$\hat{y} = z_1 + z_2 + b_{2,1}$$

Training a Multilayer Perceptron (MLP)

Let's define intermediate variables!

$$z_1 = w_{2,1,1}\, g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1})$$

$$z_2 = w_{2,1,2}\, g(x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2})$$

$$q_1 := x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1}$$

$$q_2 := x_1 w_{1,2,1} + x_2 w_{1,2,2} + \cdots + x_n w_{1,2,n} + b_{1,2}$$

$$z_1 = w_{2,1,1}\, g(q_1)$$

$$z_2 = w_{2,1,2}\, g(q_2)$$

Training a Multilayer Perceptron (MLP)

We need gradients to use the gradient method!

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{2,1}}$$

$$J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (\hat{y} - y)^2$$

$$\hat{y} = z_1 + z_2 + b_{2,1}$$

Training a Multilayer Perceptron (MLP)

We need gradients to use the gradient method!

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{2,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{2,1,1}}$$

$$J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (\hat{y} - y)^2$$

$$\hat{y} = z_1 + z_2 + b_{2,1}$$

$$z_1 = w_{2,1,1} g(x_1 w_{1,1,1} + x_2 w_{1,1,2} + \cdots + x_n w_{1,1,n} + b_{1,1})$$

Training a Multilayer Perceptron (MLP)

The derivative of the loss must be calculated w.r.t. each parameter!

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{2,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{2,1,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{1,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial q_1} \cdot \frac{\partial q_1}{\partial w_{1,1,1}}$$

• • •

Training a Multilayer Perceptron (MLP)

The derivative of the loss must be calculated w.r.t. each parameter!

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{2,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{2,1,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{1,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial q_1} \cdot \frac{\partial q_1}{\partial w_{1,1,1}}$$

. . .

It seems like a lot of computation, calculating all expressions with respect to all parameters.

Can it be done more efficiently?

Training a Multilayer Perceptron (MLP)

The derivative of the loss must be calculated w.r.t. each parameter!

$$\frac{\partial J(\Theta)}{\partial b_{2,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{2,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{2,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{2,1,1}}$$

$$\frac{\partial J(\Theta)}{\partial w_{1,1,1}} = \frac{\partial J(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial q_1} \cdot \frac{\partial q_1}{\partial w_{1,1,1}}$$

• • •

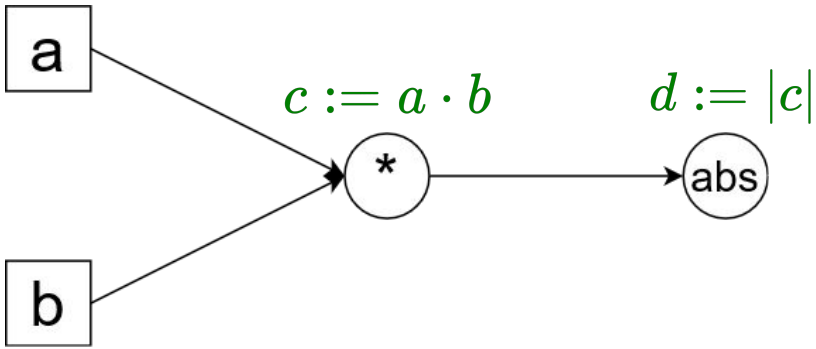
The same members appear multiple times in the derivatives. **It is useless to calculate them multiple times.**

Gradient method + keeping track of sub-results
→ **Backpropagation algorithm**

Backpropagation algorithm

Backpropagation algorithm

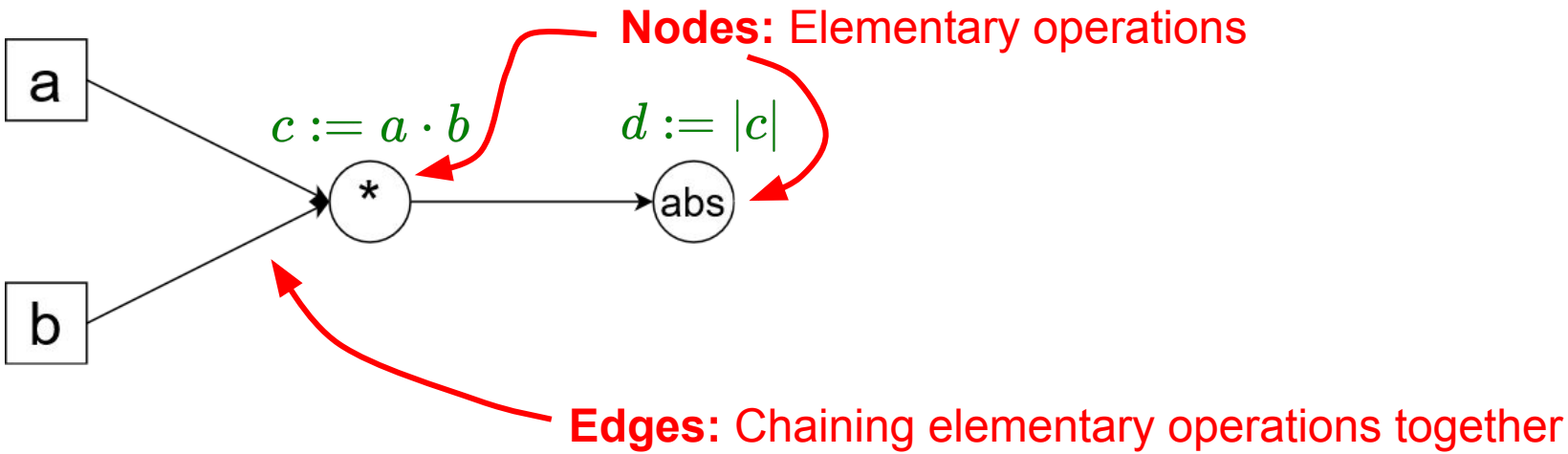
Computational graph: Representing complicated expressions



Backpropagation algorithm

Backpropagation algorithm

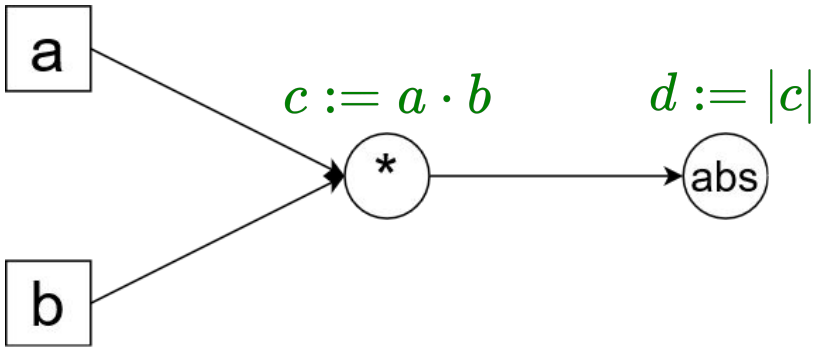
Computational graph: Representing complicated expressions



Backpropagation algorithm

Backpropagation algorithm

Computational graph: Representing complicated expressions

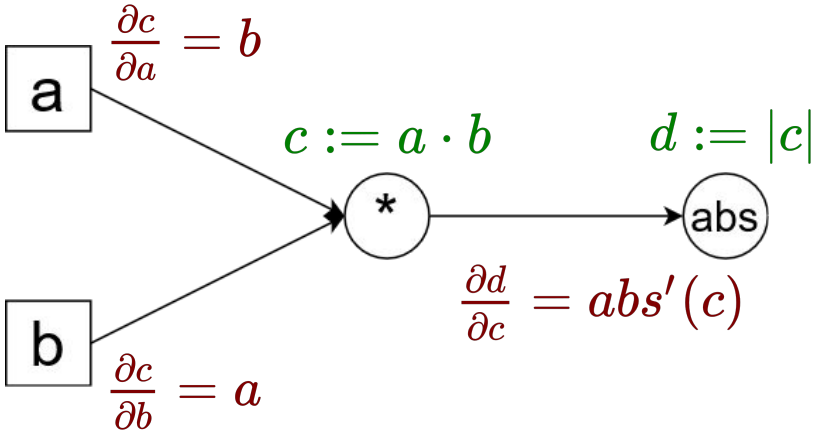


The graph tells us **what other intermediate variable need to be evaluated** in order to evaluate a given variable.

Backpropagation algorithm

Backpropagation algorithm

Computational graph: Representing complicated expressions

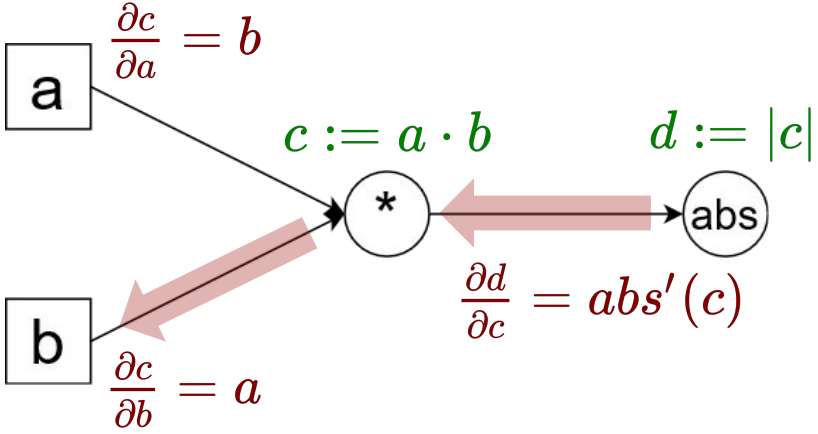


We only need to calculate **the derivative of elementary operations.**

Backpropagation algorithm

Backpropagation algorithm

Computational graph: Representing complicated expressions



$$\frac{\partial d}{\partial b} = \frac{\partial d}{\partial c} \cdot \frac{\partial c}{\partial b}$$

Then, following the chain rule, take the product of **the elementary derivatives** along the corresponding edges.

Backpropagation algorithm

Backpropagation algorithm

By linking elementary operations and defining intermediate variables, we construct a **computational graph**.

The computational graph specifies which other variables need to be evaluated in order to evaluate a given variable.

Backpropagation algorithm

Backpropagation algorithm

By linking elementary operations and defining intermediate variables, we construct a **computational graph**.

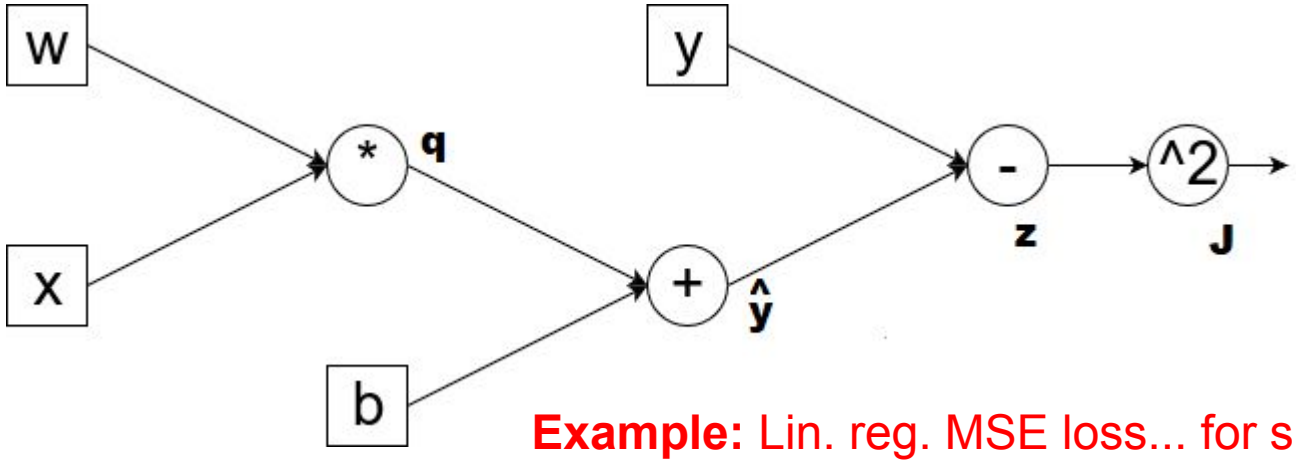
The computational graph specifies which other variables need to be evaluated in order to evaluate a given variable.

- We only calculate the **derivative of elementary operations**.
- We **take the product of the elementary derivatives** along the corresponding edges, following the chain rule.

Backpropagation algorithm

An example for a computational graph

$$J = ((wx + b) - y)^2$$



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Example: Lin. reg. MSE loss... for simplicity's sake, we have one variable, we omit the multiplication by ½, and our sample consists of a single element.

Backpropagation algorithm

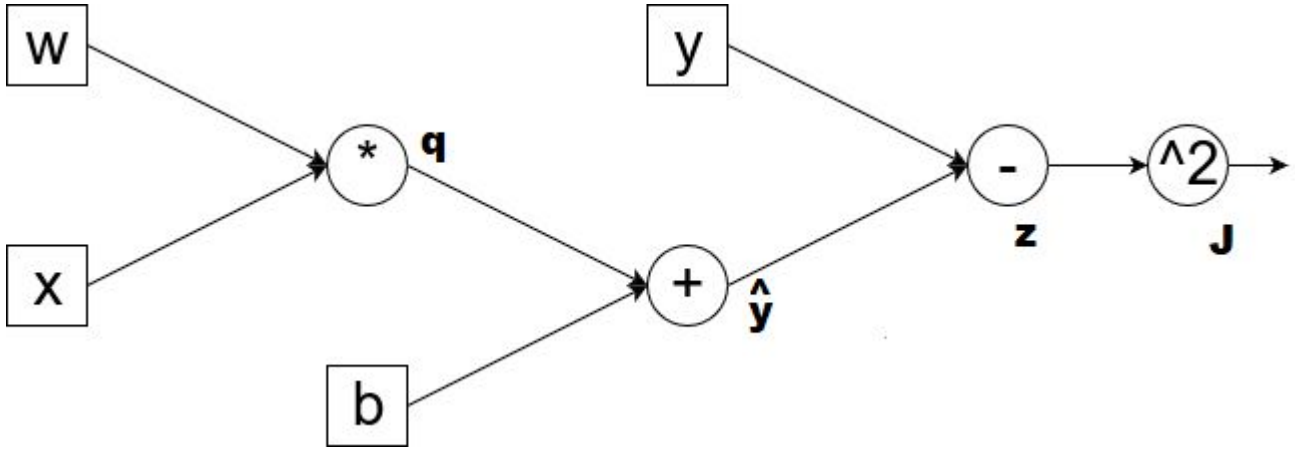
Step 1: Symbolic derivation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial b} = ? \qquad \frac{\partial J}{\partial w} = ?$$

Backpropagation algorithm

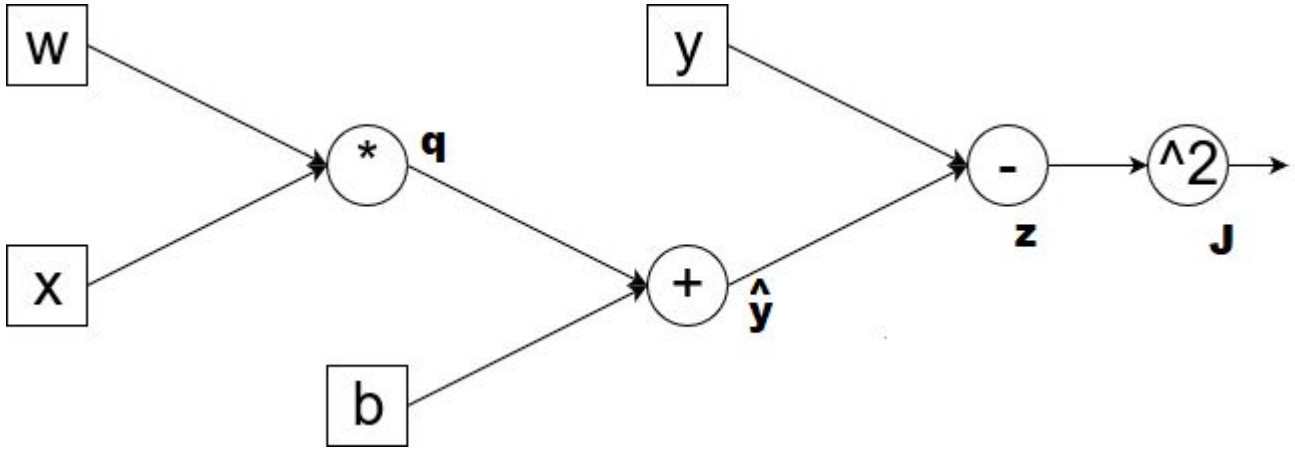
Step 1: Symbolic derivation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

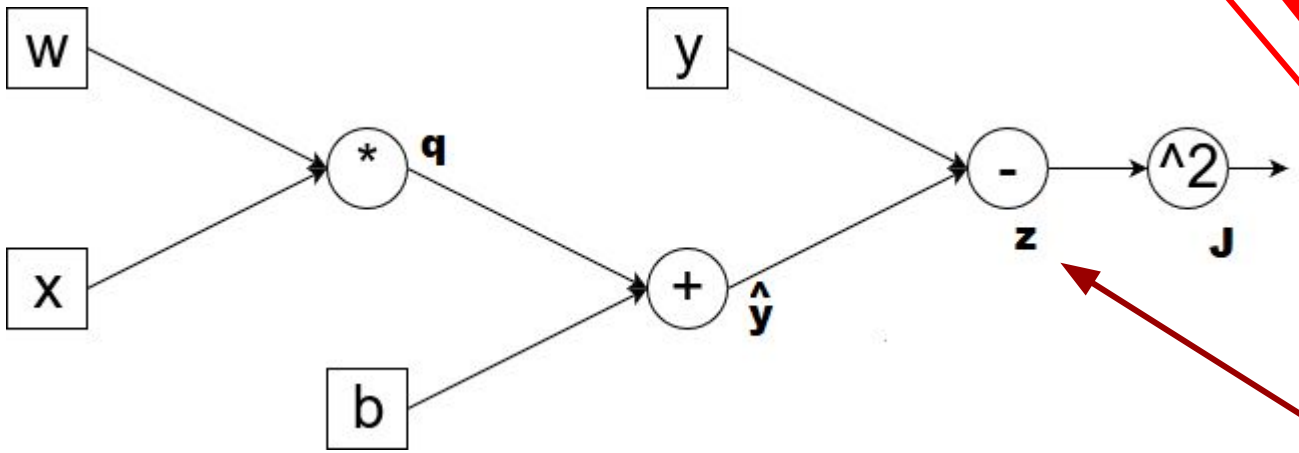
$$J = z^2$$



$$\frac{\partial J}{\partial J} = 1$$

Backpropagation algorithm

Step 1: Symbolic derivation



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

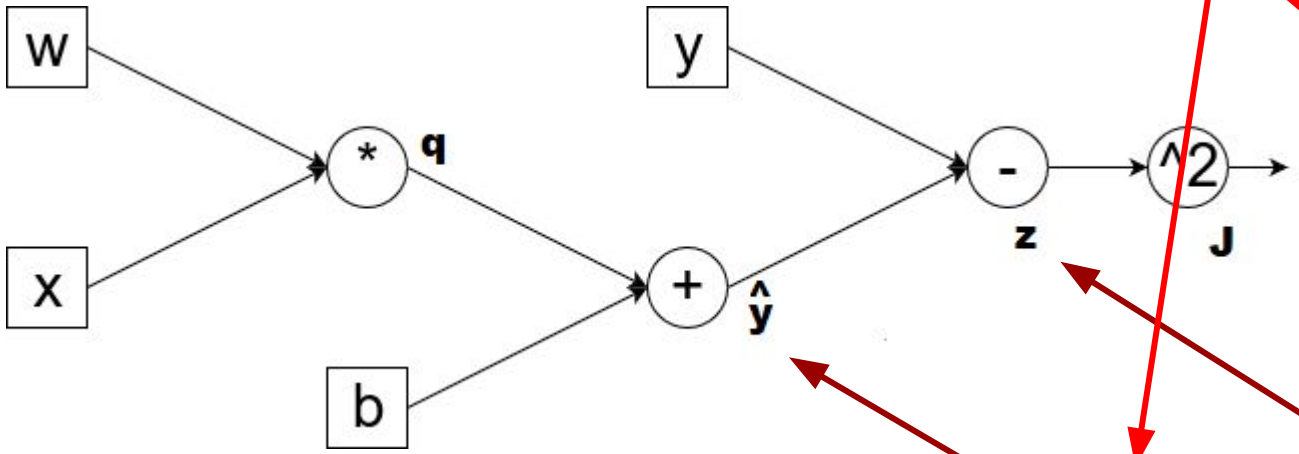
$$\frac{\partial J}{\partial z} = 2z$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

Step 1: Symbolic derivation



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

$$\frac{\partial z}{\partial \hat{y}} = -1$$

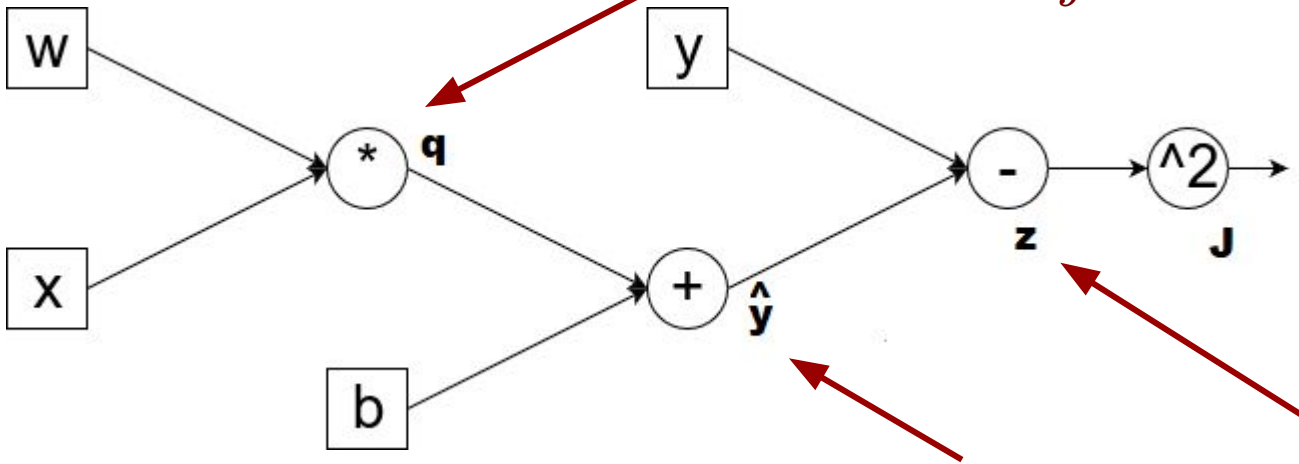
$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

Step 1: Symbolic derivation



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

$$\frac{\partial \hat{y}}{\partial q} = 1$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

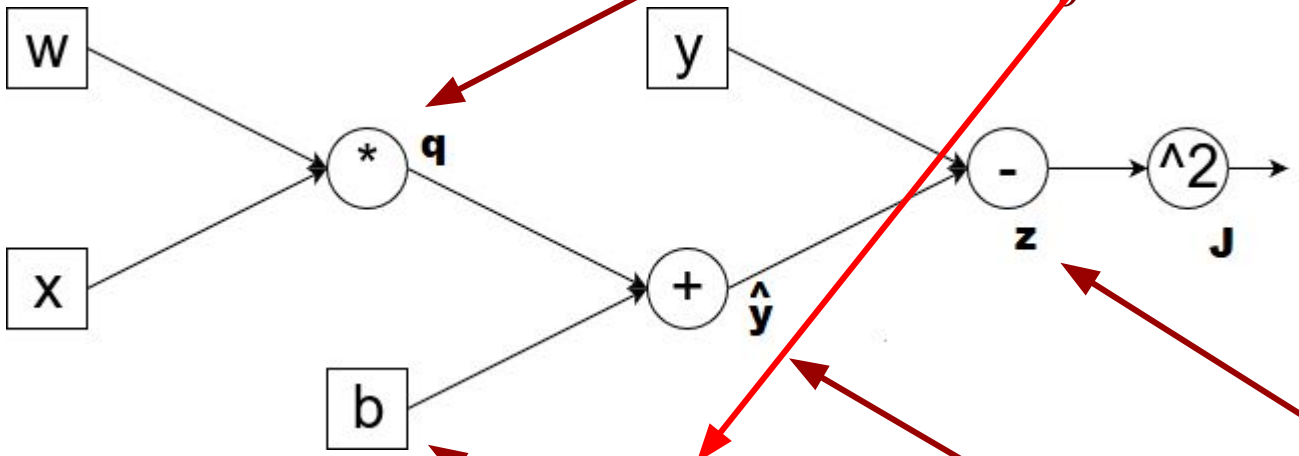
$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

Backpropagation algorithm

Step 1: Symbolic derivation



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

$$\frac{\partial \hat{y}}{\partial b} = 1$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

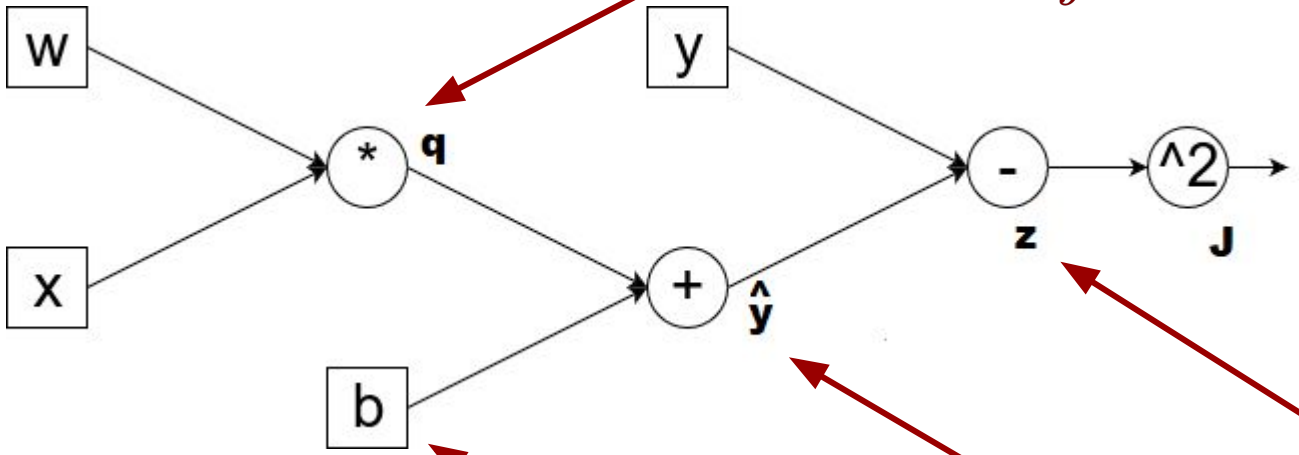
Step 1: Symbolic derivation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial q}{\partial w} = x$$

$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

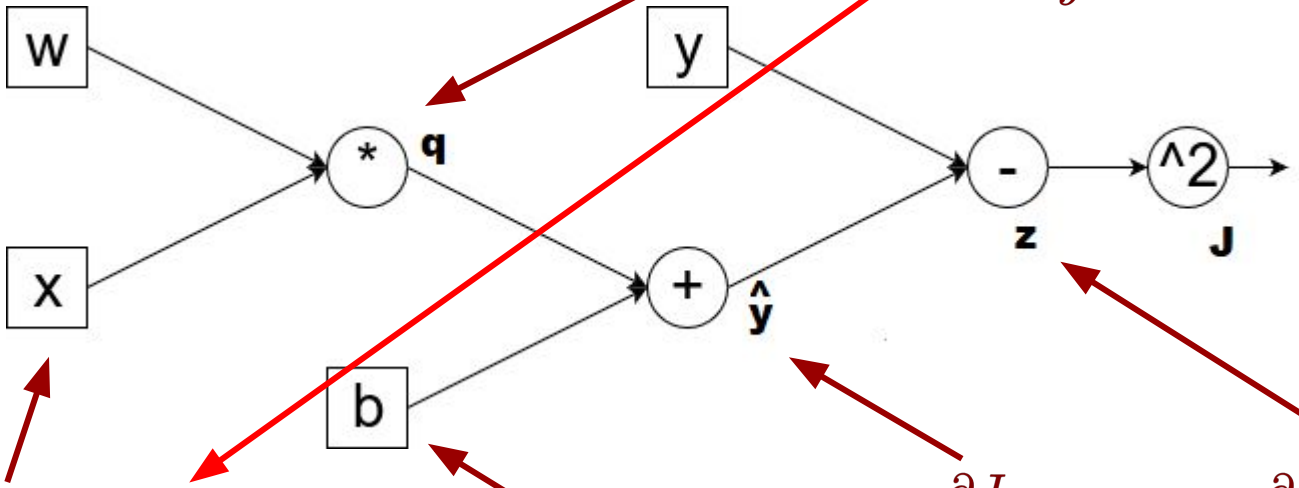
$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

Backpropagation algorithm

Step 1: Symbolic derivation



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

$$\frac{\partial q}{\partial x} = w$$

$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial x} = w \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

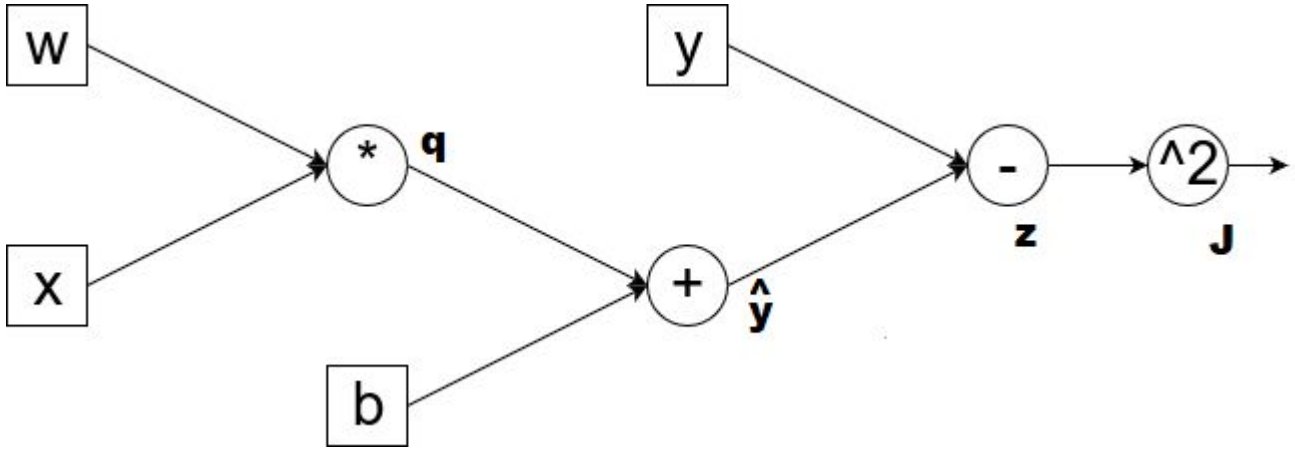
$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

Step 2: Parameter initialization

E.g., $w = 3,\ b = 4$



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Backpropagation algorithm

Select the **initial value of parameters**, for example, randomly.

Step 2: Parameter initialization

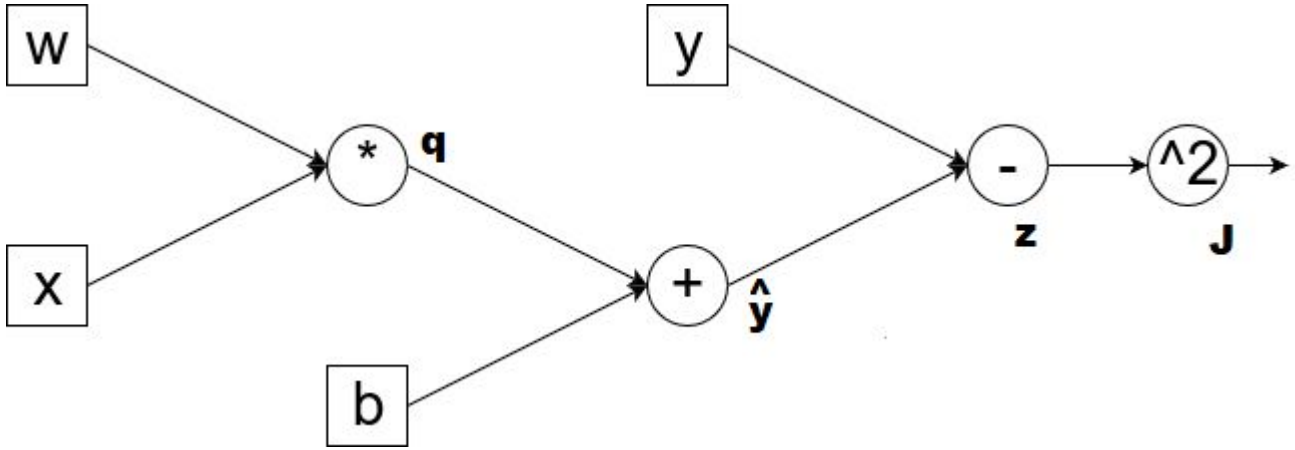
E.g., $w = 3,\ b = 4$

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

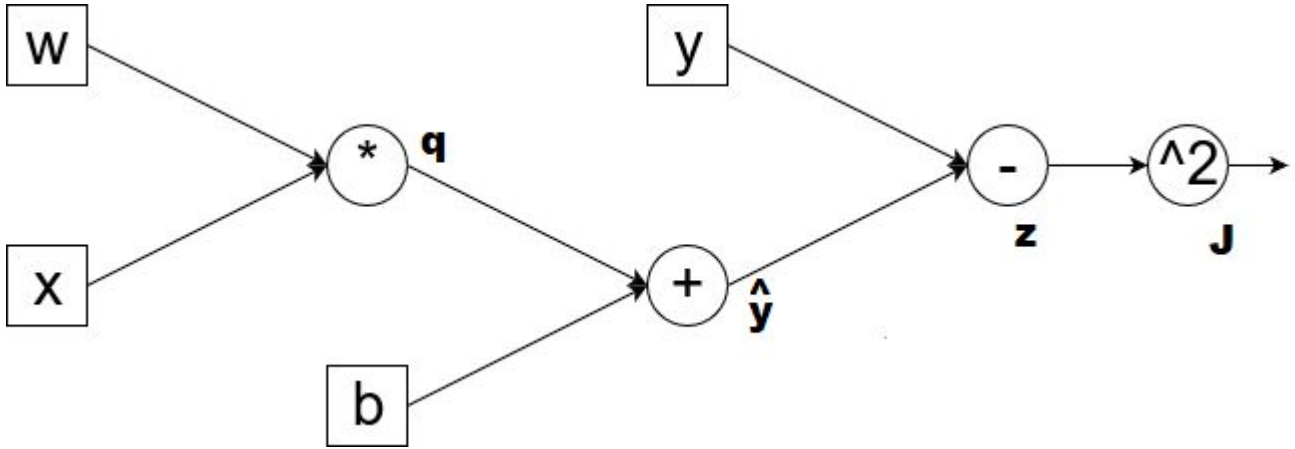
$$J = z^2$$



Backpropagation algorithm

Step 3: Choosing an example from the training set

E.g., $x = 2,\ y = 5$



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Backpropagation algorithm

In practice (SGD algorithm and variants) **a mini-batch of data is taken**, not necessarily a single example.

Step 3: Choosing an example from the training set

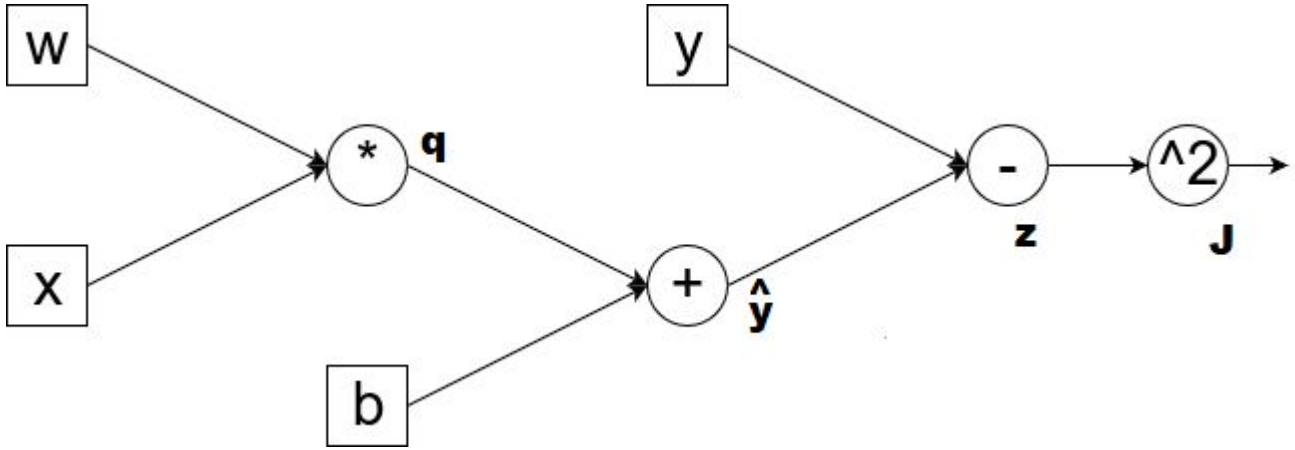
E.g., $x = 2,\ y = 5$

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

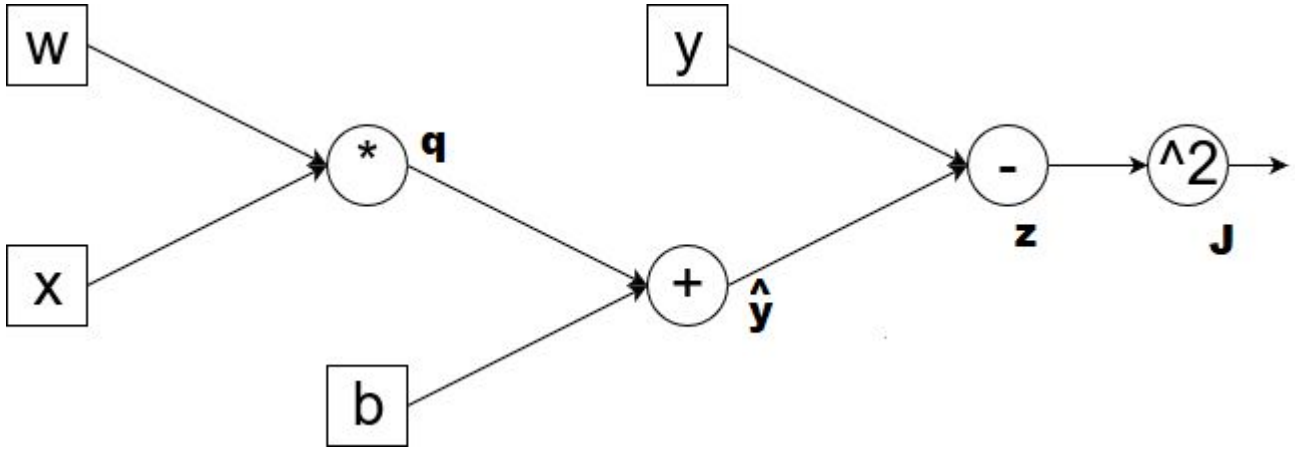
$$J = z^2$$



Backpropagation algorithm

Step 4: Substitution - feed forward

E.g., $x = 2,\ y = 5$ and $w = 3,\ b = 4$



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Backpropagation algorithm

We **substitute the input / output variables** with real data from the training set **and the parameter variables** with the current value of the parameters.

Step 4: Substitution - feed forward

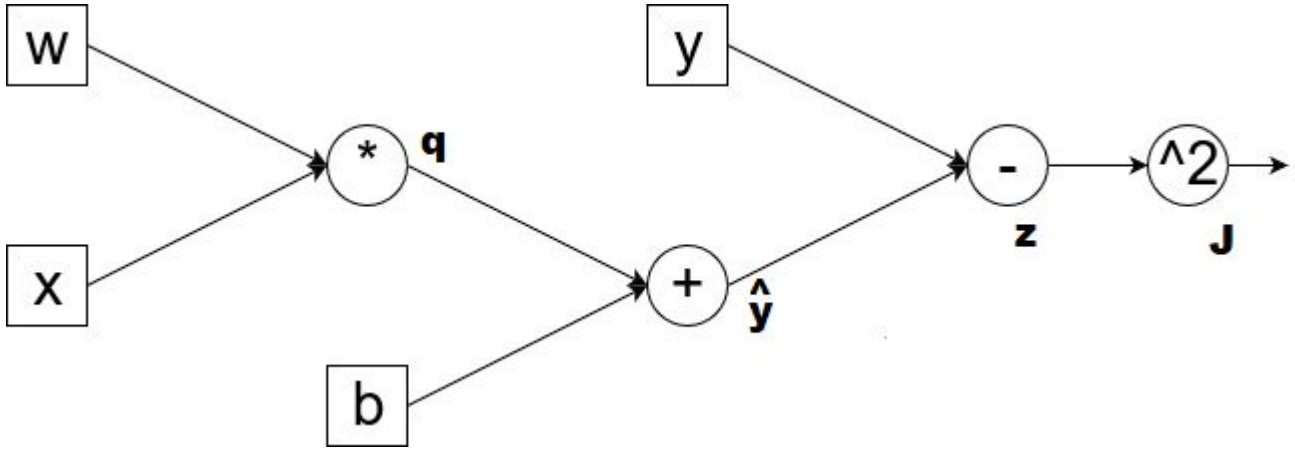
E.g., $x = 2,\ y = 5$ and $w = 3,\ b = 4$

$$q = wx$$

$$\hat{y} = q + b$$

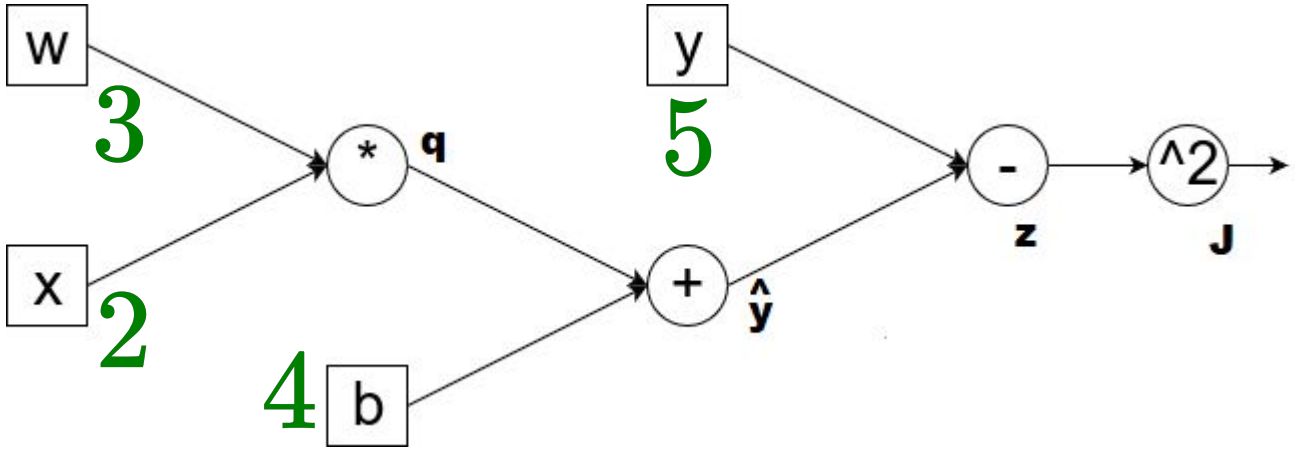
$$z = y - \hat{y}$$

$$J = z^2$$



Backpropagation algorithm

Step 4: Substitution - feed forward



$$q = wx$$

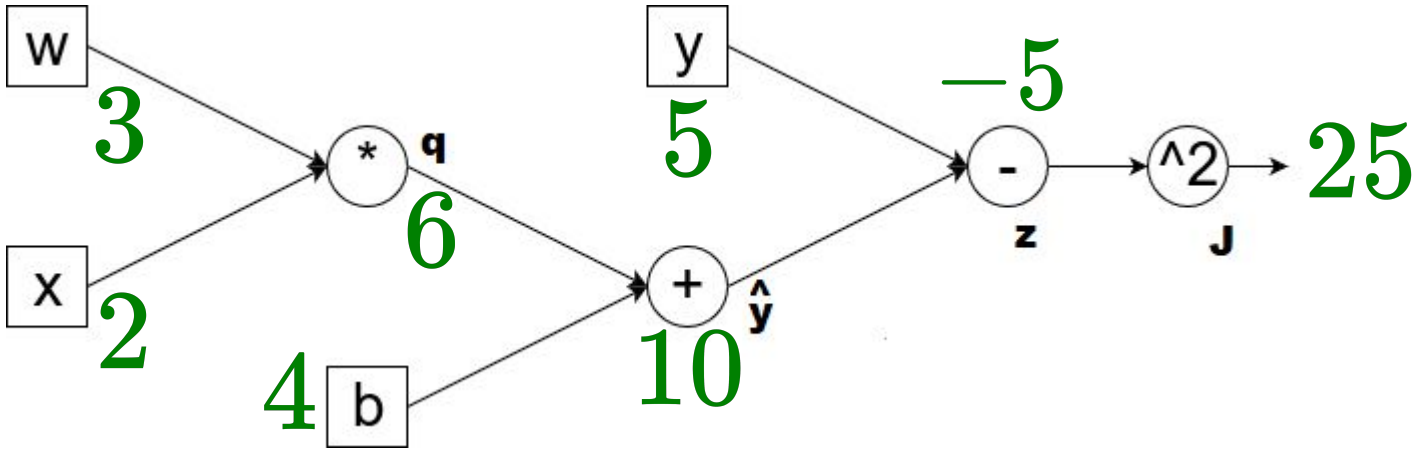
$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Backpropagation algorithm

Step 4: Substitution - feed forward



$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$

Backpropagation algorithm

Step 5: Substitution - backpropagation

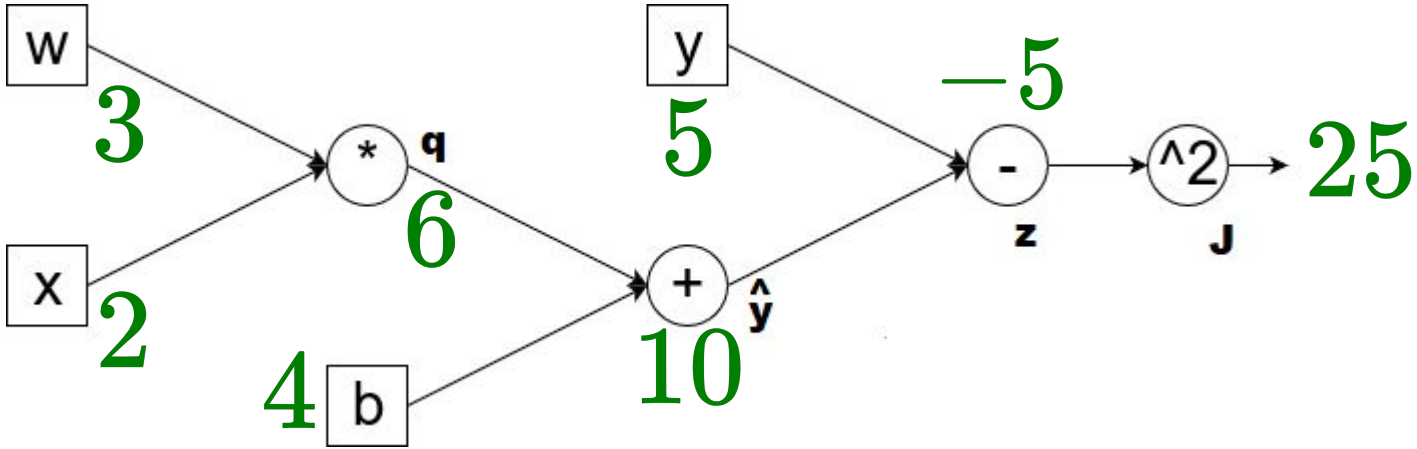
We have already calculated the gradients symbolically, now **let's substitute them in**!

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



Backpropagation algorithm

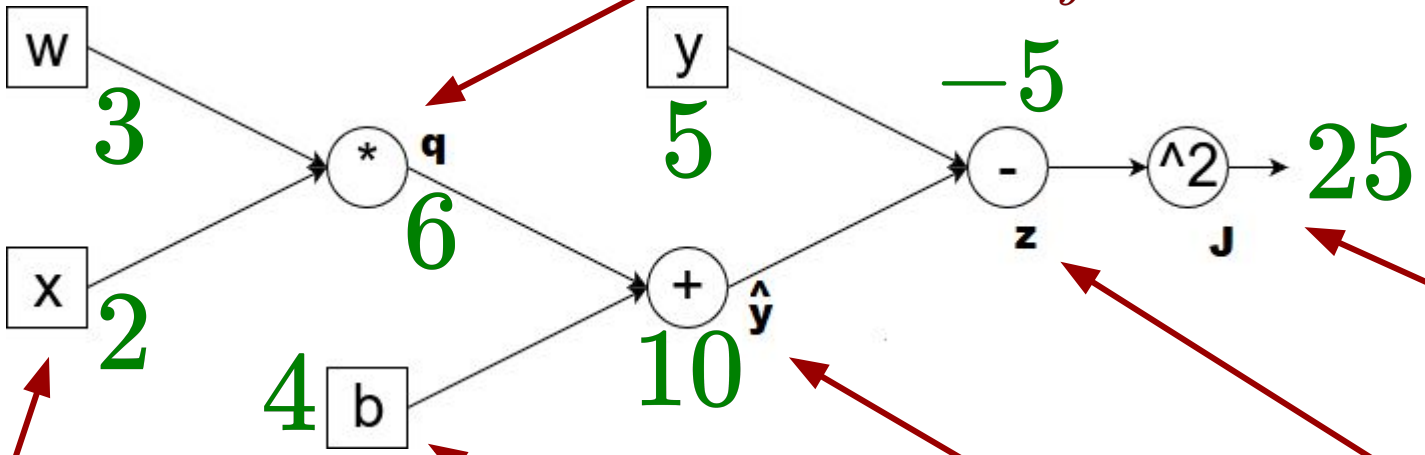
Step 5: Substitution - backpropagation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial x} = w \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

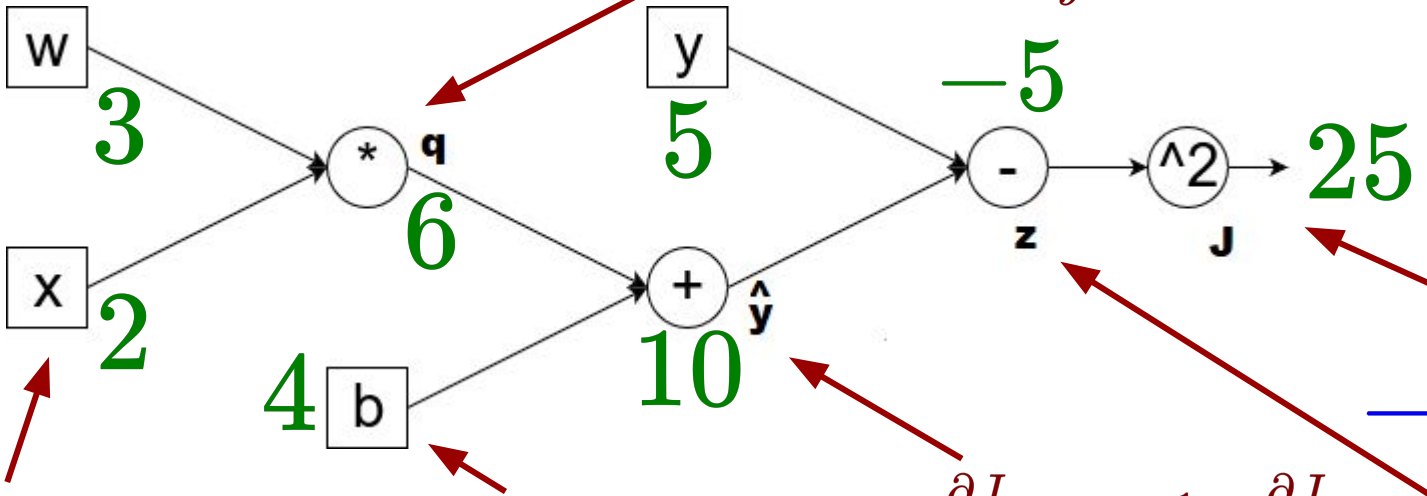
Step 5: Substitution - backpropagation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial x} = w \cdot \frac{\partial J}{\partial q}$$

1

−10

Backpropagation algorithm

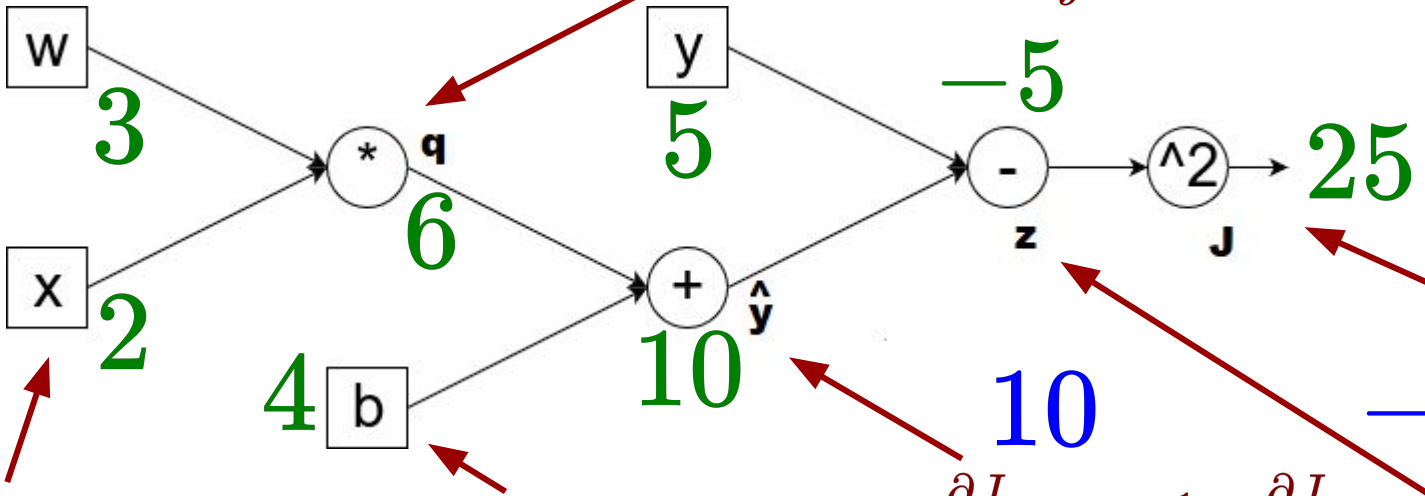
Step 5: Substitution - backpropagation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$\frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

$$\frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$\frac{\partial J}{\partial x} = w \cdot \frac{\partial J}{\partial q}$$

1

−10

10

Backpropagation algorithm

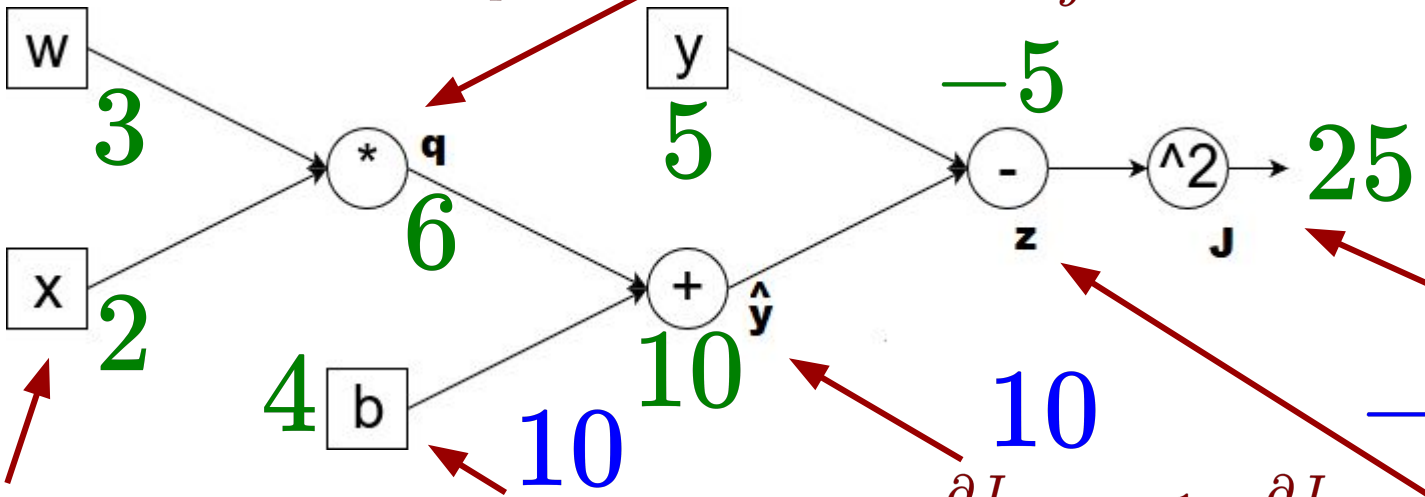
Step 5: Substitution - backpropagation

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial w} = x \cdot \frac{\partial J}{\partial q}$$

$$10 \quad \frac{\partial J}{\partial q} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$1 \quad \frac{\partial J}{\partial J} = 1$$

$$\frac{\partial J}{\partial x} = w \cdot \frac{\partial J}{\partial q}$$

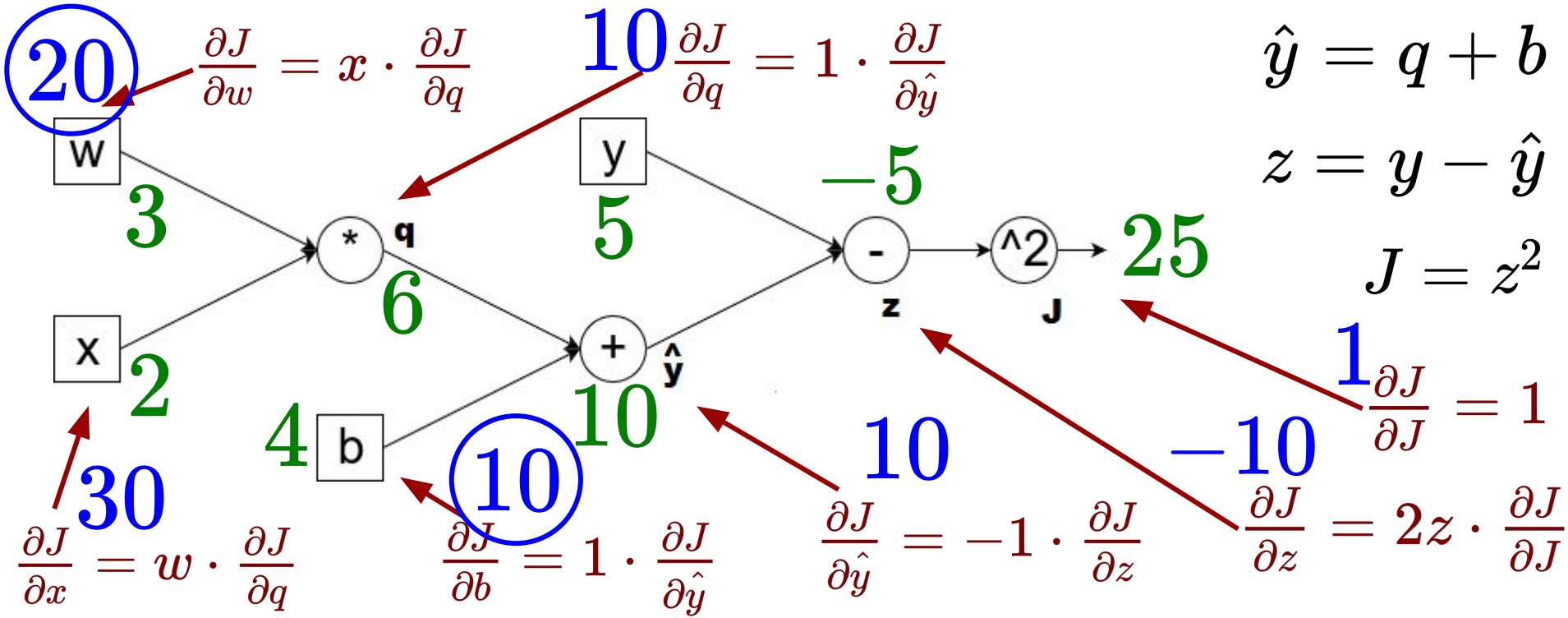
$$10 \quad \frac{\partial J}{\partial b} = 1 \cdot \frac{\partial J}{\partial \hat{y}}$$

$$10 \quad \frac{\partial J}{\partial \hat{y}} = -1 \cdot \frac{\partial J}{\partial z}$$

$$-10 \quad \frac{\partial J}{\partial z} = 2z \cdot \frac{\partial J}{\partial J}$$

Backpropagation algorithm

Step 5: Substitution - backpropagation



Backpropagation algorithm

Step 5: Substitution - backpropagation

$$\frac{\partial J}{\partial w} = 20$$

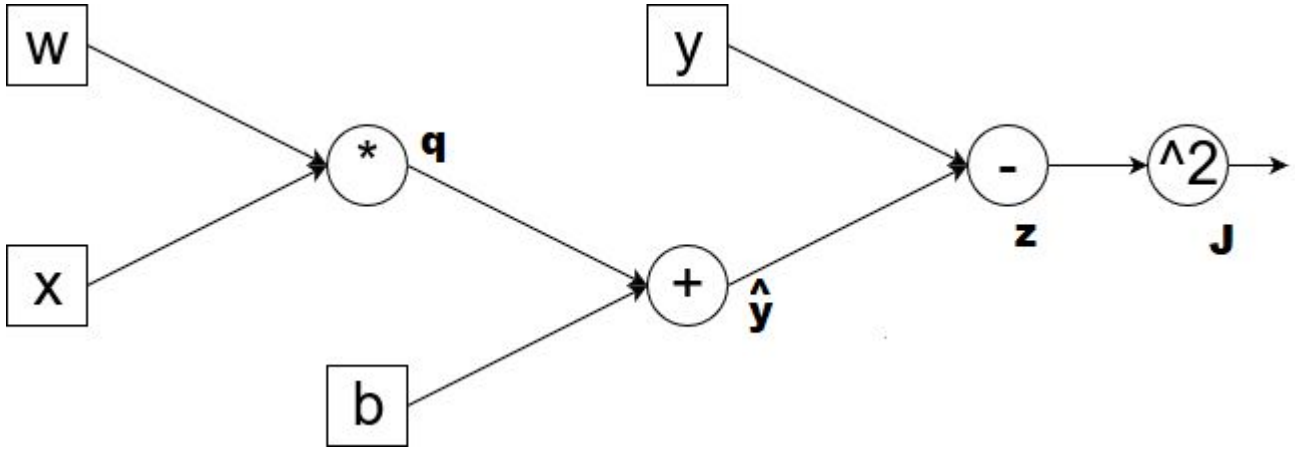
$$x = 2,\ w = 3,\ b = 4,\ y = 5$$

$$q = wx$$

$$\hat{y} = q + b$$

$$z = y - \hat{y}$$

$$J = z^2$$



$$\frac{\partial J}{\partial b} = 10$$

Backpropagation algorithm

Step 6: Updating the weights / parameters

$$x = 2,\ w = 3,\ b = 4,\ y = 5$$

$$\frac{\partial J}{\partial w} = 20 \qquad \frac{\partial J}{\partial b} = 10$$

For example, let $\alpha = 0.1$

repeat until convergence {
  for $\forall\, \theta \in \Theta$ {
    $grad_\theta = \frac{\partial}{\partial \theta} J(\Theta)$
  }
  for $\forall\, \theta \in \Theta$ {
    $\theta = \theta - \alpha\, grad_\theta$
  }
}

Backpropagation algorithm

Step 6: Updating the weights / parameters

$$x = 2,\ w = 3,\ b = 4,\ y = 5$$

$$\frac{\partial J}{\partial w} = 20 \qquad \frac{\partial J}{\partial b} = 10$$

$$w := 3 - 0.1 \cdot 20 = 1$$

$$b := 4 - 0.1 \cdot 10 = 3$$

repeat until convergence {
  for $\forall\, \theta \in \Theta$ {
    $grad_\theta = \frac{\partial}{\partial \theta} J(\Theta)$
  }
  for $\forall\, \theta \in \Theta$ {
    $\theta = \theta - \alpha\ grad_\theta$
  }
}

$$\alpha = 0.1$$

Backpropagation algorithm

Step 6: Updating the weights / parameters

$$x = 2,\ w = 3,\ b = 4,\ y = 5$$

$$\frac{\partial J}{\partial w} = 20 \qquad \frac{\partial J}{\partial b} = 10$$

$$w := 3 - 0.1 \cdot 20 = 1$$

$$b := 4 - 0.1 \cdot 10 = 3$$

repeat until convergence {
  for $\forall\, \theta \in \Theta$ {
    $grad_\theta = \frac{\partial}{\partial \theta} J(\Theta)$
  }
  for $\forall\, \theta \in \Theta$ {
    $\theta = \theta - \alpha\ grad_\theta$
  }
}

By computing the loss value with the new parameters, we can easily check whether the loss has decreased...

$$\alpha = 0.1$$

Backpropagation algorithm

Step 1: Symbolic derivation
Step 2: Parameter initialization

Step 3: Choosing an example (or a mini-batch) from the training set
Step 4: Substitution - feed forward
Step 5: Substitution - backpropagation
Step 6: Updating the parameters

GOTO STEP 3 - Stop after a given num. of iterations OR early stopping

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()` `24`
- `temperatures.ndim` `2`
- `temperatures.size()` `torch.Size([2, 12])`
- `temperatures.shape` `torch.Size([2, 12])`
- `temperatures.dtype` `torch.float32`
- `temperatures.itemsize` `4`
- `temperatures.stride()` `(12, 1)`
- `temperatures.base_` `None`
- `temperatures.device` `device(type='cpu')`
- `temperatures.requires_grad` `False`
- `temperatures.grad` `None`
- `temperatures.grad_fn` `None`

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

```
- temperatures.numel()          24
- temperatures.ndim             2
- temperatures.size()           torch.Size([2, 12])
- temperatures.shape            torch.Size([2, 12])
- temperatures.dtype            torch.float32
- temperatures.itemsize         4
- temperatures.stride()         (12, 1)
- temperatures.base_            None
- temperatures.device           device(type='cpu')
- temperatures.requires_grad    False
- temperatures.grad             None
- temperatures.grad_fn          None
```

On one hand, **PyTorch tensors are very similar to NumPy arrays…**

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()`
- `temperatures.ndim`
- `temperatures.size()`
- `temperatures.shape`
- `temperatures.dtype`
- `temperatures.itemsize`
- `temperatures.stride()`
- `temperatures.base`
- **`temperatures.device`**
- **`temperatures.requires_grad`**
- **`temperatures.grad`**
- **`temperatures.grad_fn`**

On the other hand, **PyTorch tensors are more powerful.**

- **They support** multiple types of devices, including **GPUs**.

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()`
- `temperatures.ndim`
- `temperatures.size()`
- `temperatures.shape`
- `temperatures.dtype`
- `temperatures.itemsize`
- `temperatures.stride()`
- `temperatures.base`
- **`temperatures.device`**
- **`temperatures.requires_grad`**
- **`temperatures.grad`**
- **`temperatures.grad_fn`**

On the other hand, **PyTorch tensors are more powerful.**

- **They support** multiple types of devices, including **GPUs**.
- They can be part of **computational graphs.**

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()`
- `temperatures.ndim`
- `temperatures.size()`
- `temperatures.shape`
- `temperatures.dtype`
- `temperatures.itemsize`
- `temperatures.stride()`
- `temperatures.base`
- **`temperatures.device`**
- **`temperatures.requires_grad`**
- **`temperatures.grad`**
- **`temperatures.grad_fn`**

On the other hand, **PyTorch tensors are more powerful.**

- **They support** multiple types of devices, including **GPUs**.
- They can be part of **computational graphs.**
- **`torch.autograd`** can **automatically compute gradients** over a computational graph.

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()`
- `temperatures.ndim`
- `temperatures.size()`
- `temperatures.shape`
- `temperatures.dtype`
- `temperatures.itemsize`
- `temperatures.stride()`
- `temperatures.base`
- `temperatures.device`
- `temperatures.requires_grad`
- `temperatures.grad`
- `temperatures.grad_fn`

On the other hand, **PyTorch tensors are more powerful.**

- **They support** multiple types of devices, including **GPUs**.
- They can be part of **computational graphs.**
- **`torch.autograd`** can **automatically compute gradients** over a computational graph.
- They support **lazy execution**.

PyTorch - The tensor type

```
temperatures = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temperatures.numel()`
- `temperatures.ndim`
- `temperatures.size()`
- `temperatures.shape`
- `temperatures.dtype`
- `temperatures.itemsize`
- `temperatures.stride()`
- `temperatures.base`
- `temperatures.device`
- `temperatures.requires_grad`
- `temperatures.grad`
- `temperatures.grad_fn`

On the other hand, **PyTorch tensors are more powerful.**

- **They support** multiple types of devices, including **GPUs**.
- They can be part of **computational graphs.**
- **`torch.autograd`** can **automatically compute gradients** over a computational graph.
- They support **lazy execution**.

→ **More details in the practice materials!**